4. Сеньор 1
4.1. Серверная разработка на Java 14
4.1.1.1. Старт проекта и инженерное окружение
100%
Выполнено 5 из 5
4.1.1.2. От L1-сценария социальной сети к API
100%
Выполнено 5 из 5
4.1.2.1. Хранение данных со Spring Data JPA
60%

Публикации и пользовательская лента [#505784]

2 мин.

Новая Alexander Vasiliev Детали

0 0

Мы уже реализовали границу хранения для предложений дружбы, друзей и подписок. Но социальная сеть также должна хранить публикации и формировать пользовательскую ленту. В этом уроке мы спроектируем таблицу `posts`, подготовим миграции и реализуем репозитории публикаций и ленты через `JdbcTemplate`.

При проектировании важно разделить две задачи. `PostRepository` изменяет и читает отдельные публикации, а `FeedRepository` строит список опубликованных записей авторов, на которых подписан пользователь. Лента не требует отдельной таблицы на этом этапе: она формируется SQL-запросом из `subscriptions` и `posts`.

Процесс публикации

Выполнено 3 из 5

- Модель данных социальной сети [#505780 #567308]
  Выполнена 0
- JPA-сущности и репозитории [#505781 #567310]
  Выполнена 0
- Тестирование репозитория с Testcontainers [#505782 #567327]
  Выполнена 0
- Запросы подписчиков и друзей [#505783 #567329]

На этом этапе пользователь может:

- создать публикацию;
- получить собственную или доступную публикацию;
- изменить свою публикацию;
- удалить свою публикацию логически.

Таблица `posts` содержит:

Копировать

```
id
author_id
title
content
status
```


Новая 0

- 4.1.2.2. REST API как внешний адаптер

Выполнено 0 из 7

- 4.1.3.1. Доменная модель и границы ответственности

Выполнено из

Сервис не хранит таблицу пользователей, поэтому `author_id` содержит UUID пользователя из Keycloak без внешнего ключа. Ограничение `CHECK` допускает состояния `PUBLISHED` и `DELETED`.

Право изменения будет полноценно защищено после подключения Keycloak, но контракт репозитория уже должен принимать `actorId`. SQL обновления обязан включать автора в условие `WHERE`, чтобы один пользователь не мог изменить публикацию другого.

Индекс `(author_id, created_at DESC, id DESC)` поддерживает получение публикаций конкретного автора в стабильном порядке.

4.1.3.2. Ошибки и единый контракт отказов

Выполнено из

4.1.3.2. Транзакции, консистентность и конкуренция

Выполнено из

4.1.3.3. Интеграционные и сценарные тесты

Выполнено из

4.1.4.1. Структурные логи и корреляция

Выполнено из

4.1.4.2. Метрики процесса

Выполнено из

4.1.4.3. Распределённые трассы

Выполнено из

4.1.5. Асинхронные и периодические процессы

Создайте индекс отдельной миграцией и добавьте rollback как для таблицы, так и для индекса.

Что такое лента

Первый вариант ленты строится при чтении:

Копировать

```
текущий пользователь
  → его подписки
  → опубликованные посты авторов
  → сортировка от новых к старым
  → страница результата

```

В ленту не входят удалённые публикации и публикации пользователей, на которых текущий пользователь не подписан.

Запрос ленты

Запрос должен соединить `subscriptions` и `posts`, отфильтровать подписки по `follower_id`, оставить `PUBLISHED` и применить стабильную сортировку:

Копировать

```
SELECT p.id,
       p.author_id,
       p.title,
       p.content,
       p.status,

```

Выполнено из

4.1.6. Развертывание и эксплуатация

Выполнено из

4.1.7. Итоговый проект

Выполнено из

```
       p.created_at,
       p.updated_at
FROM subscriptions s
JOIN posts p ON p.author_id = s.followed_id
WHERE s.follower_id = ?
  AND p.status = 'PUBLISHED'
ORDER BY p.created_at DESC, p.id DESC
LIMIT ? OFFSET ?;

```

Идентификатор используется вторым ключом сортировки, потому что несколько публикаций могут иметь одинаковое время.

Для первой реализации используйте параметры `limit` и `offset` в запросе репозитория. В документации отметьте ограничения offset pagination на больших и быстро меняющихся лентах. Cursor pagination будет отдельным развитием процесса.

Проверяем связь процессов

До подписки пост автора отсутствует в ленте читателя. После подписки появляется. После отписки снова исчезает. После принятия предложения дружбы оба друга видят новые публикации друг друга, потому что процесс создал взаимные подписки.

Именно эта проверка связывает SQL-запрос с бизнес-процессом, а не оставляет его отдельным упражнением по работе с базой.

Задание

1. Создайте SQL-миграцию таблицы `posts` с первичным ключом, обязательными полями, допустимыми состояниями `PUBLISHED` и

`DELETED`, а также блоком `--rollback`.

2. Создайте отдельную миграцию индекса `(author_id, created_at DESC, id DESC)` и добавьте для неё rollback.
3. Добавьте `PostEntity`, `PostRepository` и его реализацию через `JdbcTemplate`.
4. Реализуйте в `PostRepository` создание, чтение, изменение и логическое удаление публикации.
5. Создайте `FeedRepository` и реализуйте SQL-запрос ленты через соединение `subscriptions` и `posts`.
6. Передавайте в запрос ленты `limit` и `offset` и используйте стабильную сортировку `created_at DESC, id DESC`.
7. Добавьте интеграционные тесты `JdbcPostRepositoryTest` и `JdbcFeedRepositoryTest` на PostgreSQL Testcontainers.
8. Проверьте появление публикации в ленте после подписки, её исчезновение после отписки и исключение публикаций в состоянии `DELETED`.
9. Добавьте изменения в репозиторий и пришлите ссылку на коммит.

Публикации и пользовательская лента [#505784]

Недавно решали

Бухтояров Андрей[#378035] 13.10.2026 13:28

Отправить на проверку | К списку | Предыдущая | Решения: 1

Понравилось задание?

Поставьте понравилось или нет. Эта оценка позволяет нам улучшать проект Job4j.

0 0

Обсуждения

Эти комментарии доступны всем ученикам. Поделитесь своим мнением о зада че, полезными ссылками, рекомендациями по решению. Не скидывайте решен ия целиком!

Загрузить: 0 Обсудить

© Job4j. Все права защищены.